



# AI: DEMYSTIFYING THE ALGORITHM

## Large Language Model overview

Presentation to Presidio of Monterey

24 August 2026

# Agenda

Purpose: Intro workforce to fundamental concepts of Large Language Models.

| ORDER | INFORMATION                                       |
|-------|---------------------------------------------------|
| 01    | TDAC Overview                                     |
| 02    | Neural Networks                                   |
| 03    | Tokens (and next token prediction)                |
| 04    | Embedding vector                                  |
| 05    | Tokens (and next token prediction)                |
| 06    | Large Language <b><u>Model</u></b> vs Application |
| 07    | Context (and context window)                      |
| 08    | Retrieval Augmented Generation (RAG)              |
| 09    | AI slop (work slop)                               |
| 10    | Bias and manipulation                             |

This will be a lot of information!

The purpose is just to introduce these concepts to you, not to make you an expert.

(there's no exam 😊 )

Hopefully, this gives you a lot to think about, ask questions about, and learn more about later.

# TDAC Advanced Analytics Branch



Conducting applied research at NPS to support analysis for the Army.



T2COM



FCC

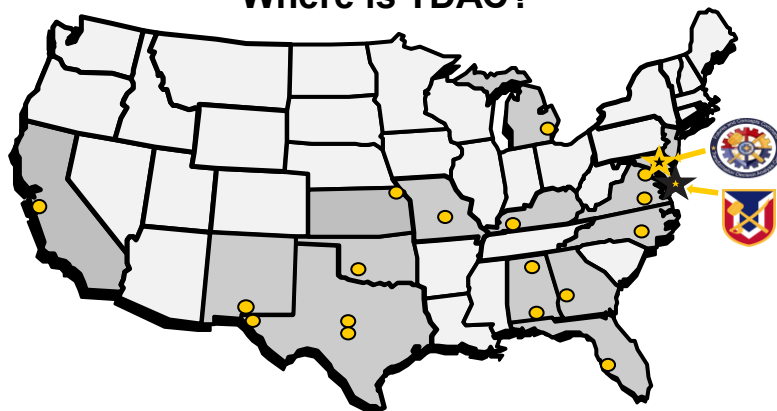


TDAC

The Transformation Decision Analysis Center (TDAC) is part of **Futures and Concepts Command (FCC)**, under **Transformation and Training Command (T2COM)**.

**MISSION:** Lead, conduct, and deliver relevant, credible, objective, and timely analysis in support of Army decision needs.

Where is TDAC?



## TDAC Advanced Analytics Branch

**MISSION:** Develop methods, models, and tools that enable TDAC Analysts to do their jobs more effectively.

**CONCEPT:** The TDAC Advanced Analytics Branch is located at the Naval Postgraduate School (NPS) to partner with students and faculty conducting applied research.

### Chief

LTC Joe Pedersen  
PhD, RPI

### Deputy

Dr. Brian Wade  
PhD, GT

### Operations Research Analysts

MAJ David Allen  
MS, NPS

MAJ Jonathan Kasprisin  
MS, USC

MAJ Austin McGahan  
MS, NPS

MAJ Ryan Hensel  
MS, CU

### Administration & Budget

Ms. Sandra Lackey  
Admin Support

# Artificial Intelligence



## How can computers help us with “prediction” tasks?

### ■ Example “prediction” tasks:

1. Given a medical image, predict if there is a tumor present (if so, predict its location in the image)

**Input:** image **output:** probability (0 to 1)

**Input:** image **output:** location of bounding box

2. Given an image of text, predict the characters (called Optical Character Recognition or OCR)

**Input:** image **output:** location of each character

**Input:** image **output:** probability of each of character

3. Given facts about a driver and their car(s) covered, predict the expected payouts for insurance claims

**Input:** driver demographics, driving history, car details **output:** dollar amount

4. Given a person’s credit history, predict if they will default on a particular loan.

**Input:** credit history and loan details **output:** probability (0 to 1)

5. Given a person’s Netflix history (movies watched and/or rated), predict how they would rate a movie.

**Input:** movie watching/rating history and facts about move **output:** predicted rating

# Artificial Intelligence: Expert Systems

Expert systems were human coded rules learned from experts

## ■ Example

1. Given a medical image, predict if there is a tumor present (if so, predict its location in the image)

**Input:** image **output:** probability (0 to 1)

**Input:** image **output:** location of bounding box

For example, expert system designers could interview doctors to learn what they look for in a medical image to determine if there is a tumor, then try to write a computer program that performs the same process.

Do all doctors follow the same process?

Would it be easy for any doctor to describe his/her process in the detail necessary to write a program?

Translating human expertise into a computer algorithm is tedious, time consuming, and usually not very accurate for tasks without explicitly codified rules

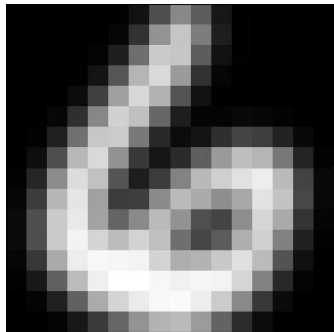
# Artificial Intelligence: Machine Learning

Can we use data (not experts) to find a **function** that predicts accurately?

- Example: Given an image of a number 0-9, predict which number it is.

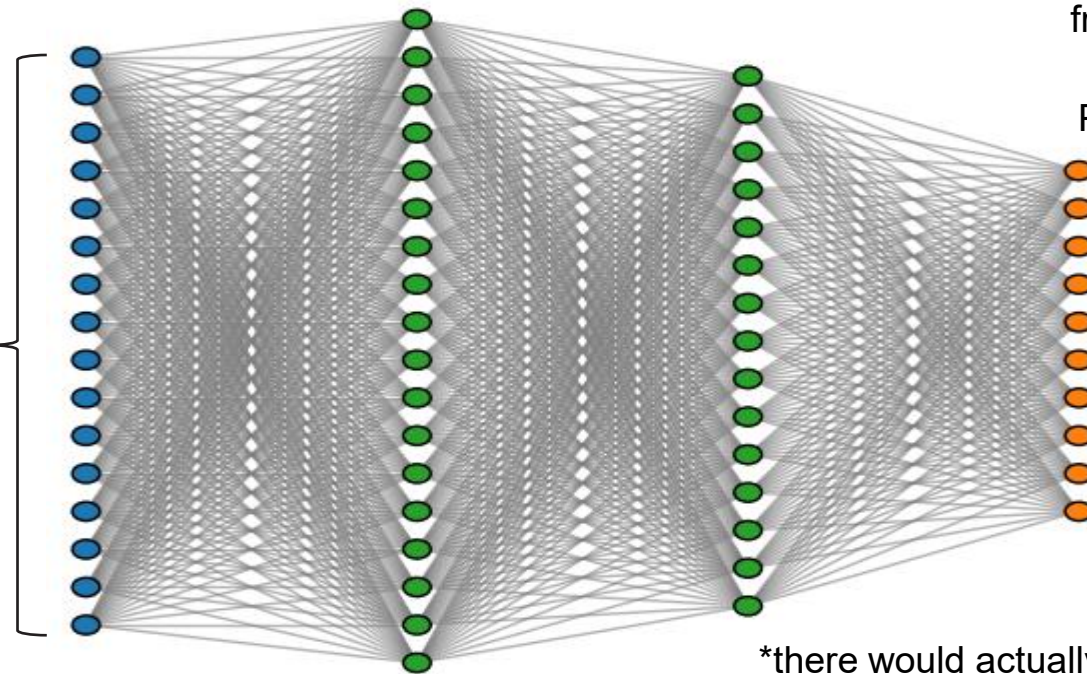
Input: image      output: probabilities (0 to 1)

Each pixel value becomes  
an input (blue circle)\*



Pixel values  
are the input

0 = black  
0 < grey < 1  
1 = white



Each green or orange node is computed  
from the values of the nodes to the left of it

Probabilities are the output

|        |               |
|--------|---------------|
| 0.0018 | character "0" |
| 0.0001 | character "1" |
| 0.0002 | character "2" |
| 0.0003 | character "3" |
| 0.0000 | character "4" |
| 0.0061 | character "5" |
| 0.9827 | character "6" |
| 0.0001 | character "7" |
| 0.0078 | character "8" |
| 0.0010 | character "9" |

The model  
predicts a 98%  
chance the  
image is a "6"

\*there would actually be way more nodes

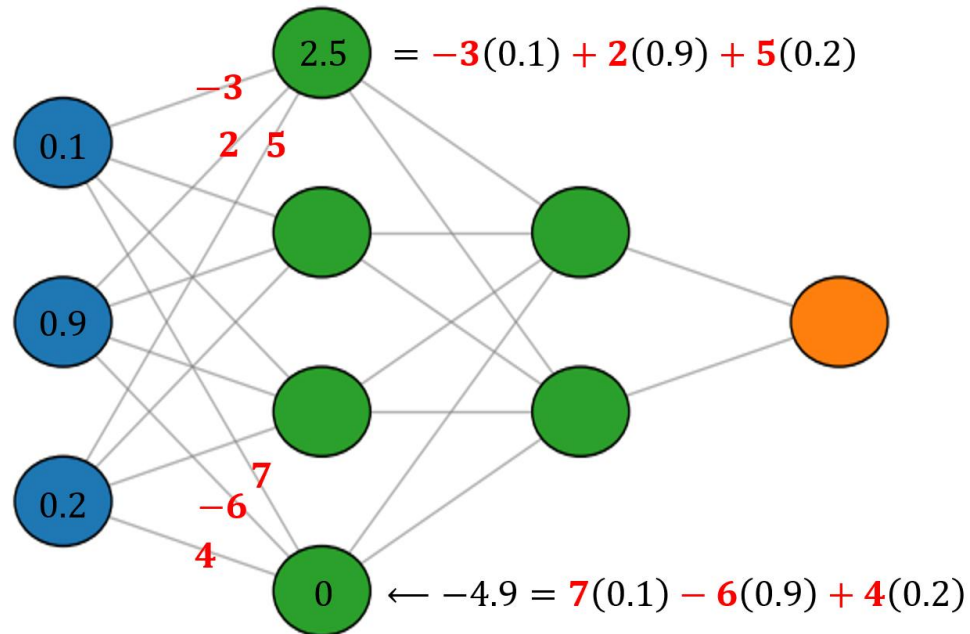
For any task, there is a **neural network** that can predict the training data accurately.

The real goal is making a model that makes accurate predictions on **new data**.

# Artificial Intelligence: Neural networks

Neural network math is not complicated (and the computer does it 😊)

This is an unrelated, small neural network, just to illustrate how the math works.



- The value of each node (circle) is computed in three steps:
  - **MULTIPLY** the values of the nodes to the left of it by the values of the **weights** (the lines connecting them).
  - **ADD** those products together
  - **IF** that sum is positive, use that value, **ELSE** use zero
- For complicated prediction tasks, the neural networks contain BILLIONS (or TRILLIONS) of weights
- Finding numbers that “work” (predict accurately on new data) for large neural networks takes:
  - Enough training data
  - Powerful enough computers
  - For LLMs, months & millions of dollars of electricity

Once we find numbers that “work”, we use those numbers. That’s the neural network – numbers. The neural network doesn’t change, or “think”, or have feelings, or intentions. It’s not Skynet.

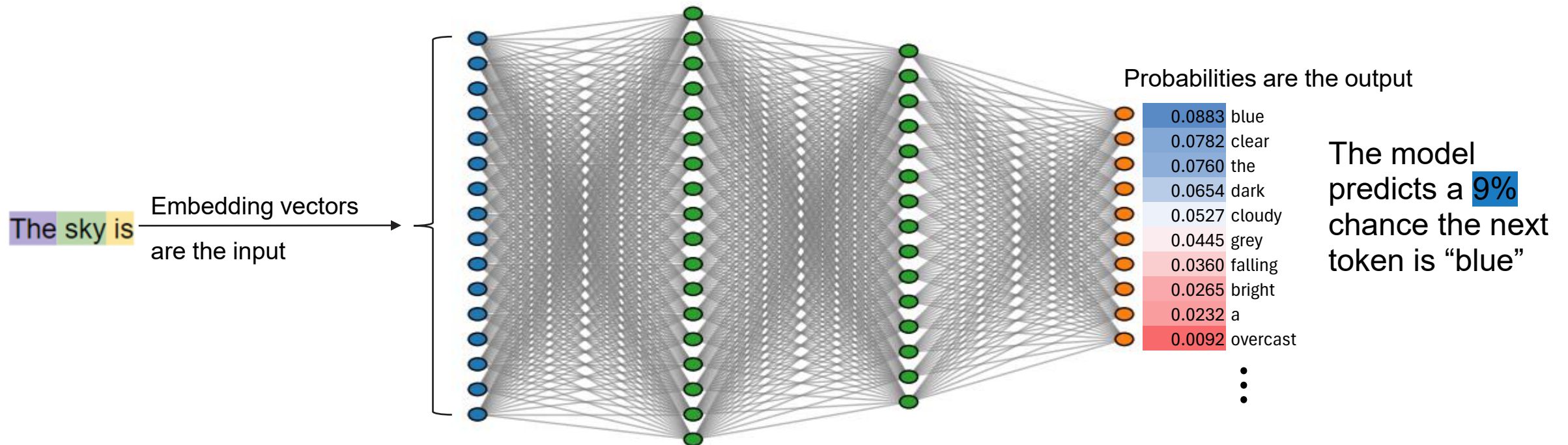


# Large Language Models: Neural networks

An LLM is just a neural network that predicts the next “token”

**Input:** an “embedding vector” (list of numbers) for each “token”

**output:** for every possible “token”, the probability (0 to 1) that it comes next



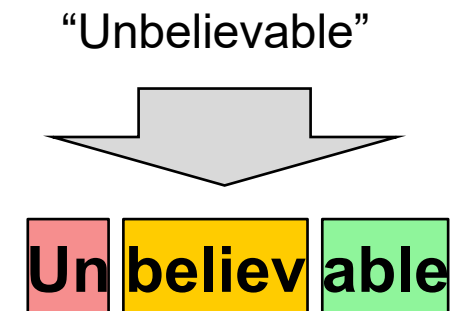
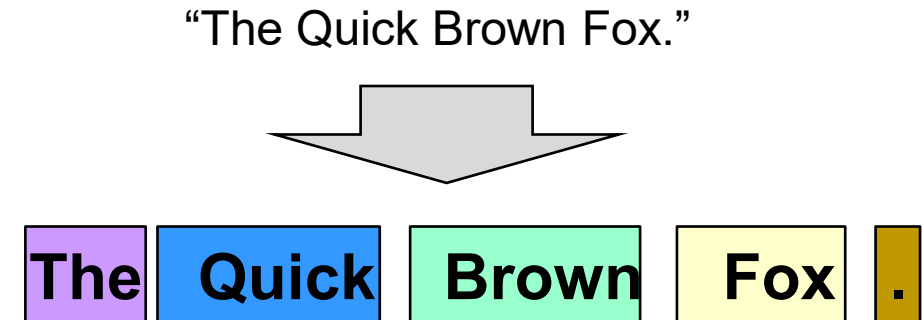
This is an incredibly oversimplified image but depicts the main ideas. An actual LLM has a much more complicated architecture, with BILLIONS (or trillions) of weights.



# Large Language Models: Tokens

A **token** is the fundamental unit for LLMs. The input to an LLM is first “tokenized”.

- Tokens are fundamental units of input that LLMs process.
- These can be:
  - Whole words (e.g., “hello”)
  - Parts of words (e.g., “ing”, “un”)
  - Punctuation marks (e.g., “.”, “?”)
- Tokenization is the process of breaking down raw text into tokens.
- Multimodal models also break other media (images, video, audio) into “tokens”.
- Some LLM service providers charge “per token” or charge a flat fee but with a “token limit”. These can count input tokens and/or output tokens.

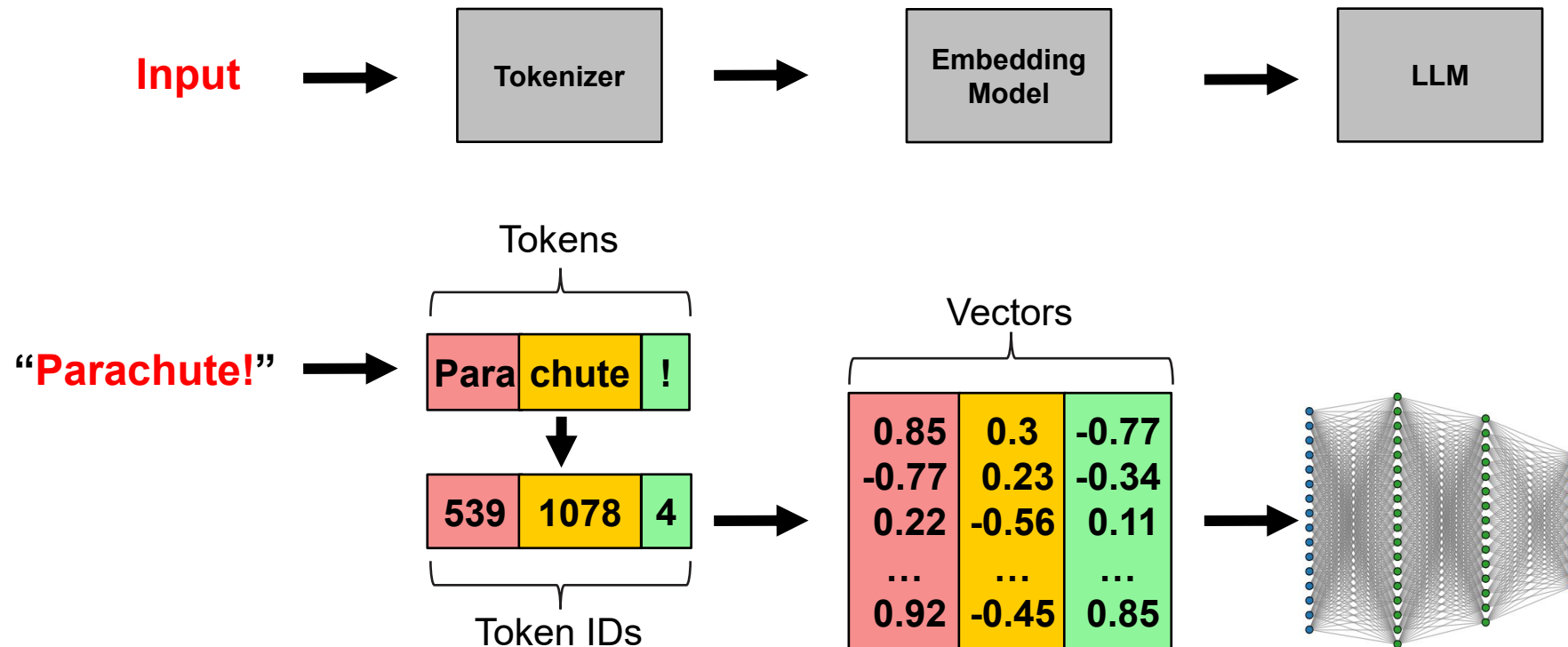


For text, a **token** is a word, part of a word, punctuation, or special tokens like <|end|>.

# Large Language Models: Embedding Vectors

The input to a neural network needs to be numbers.

- After the input is tokenized, each token is replaced by a vector (list of numbers) that represents its meaning.



The embedding vectors of text that are similar in meaning are near each other

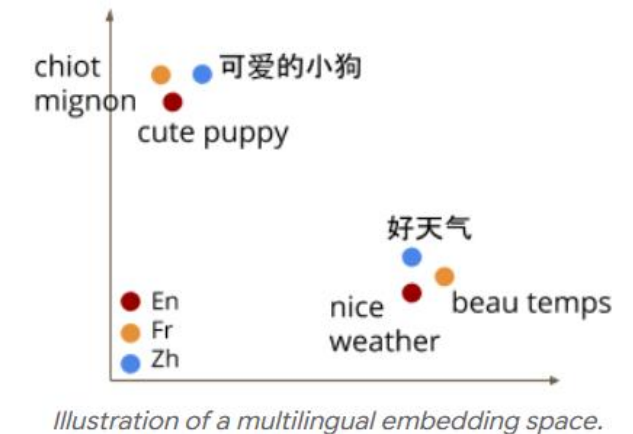
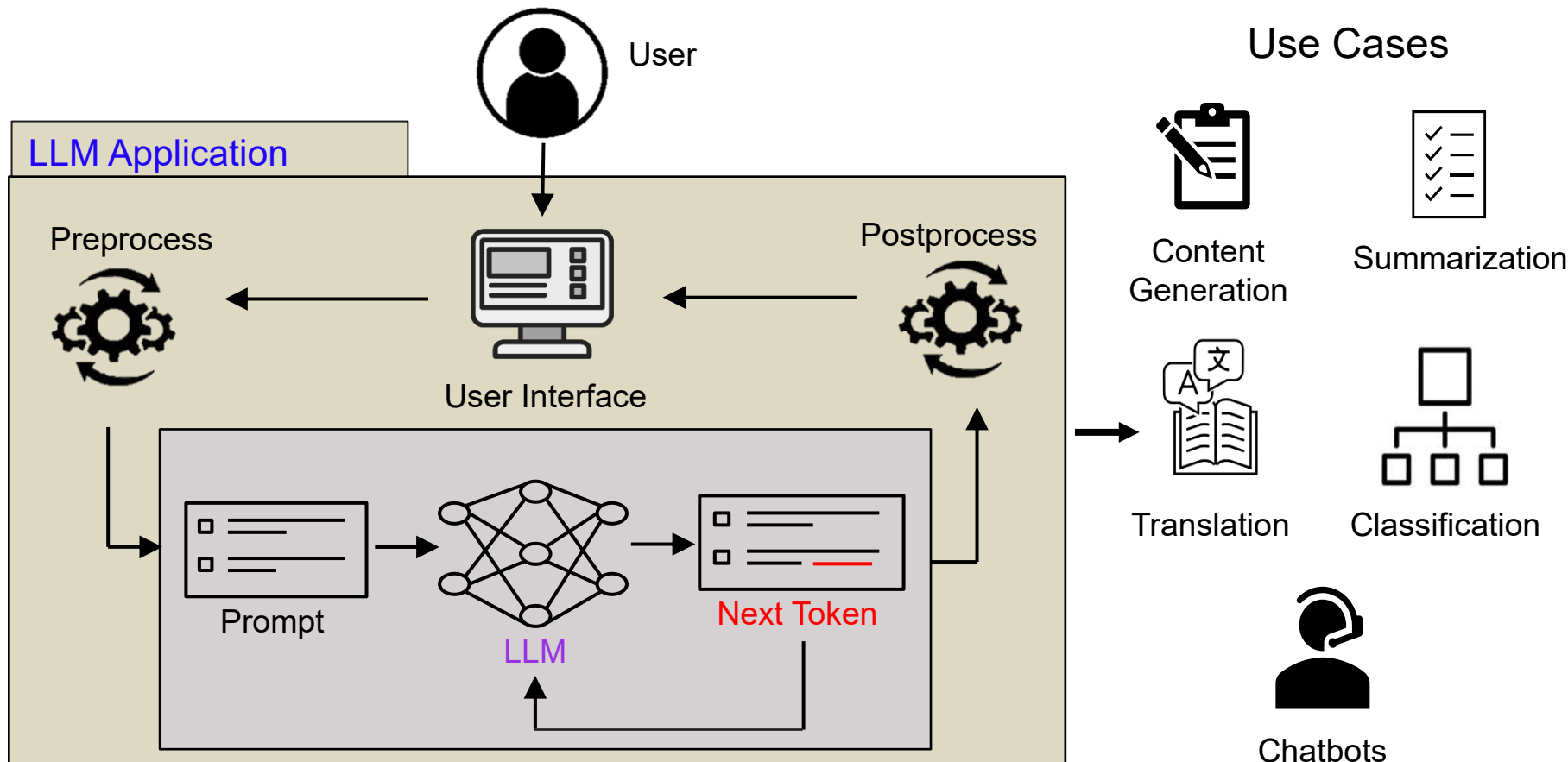


Image retrieved on August 23, 2026, from <https://research.google/blog/language-agnostic-bert-sentence-embedding/>

Embedding vectors represent the meaning of text in the sense that the more closely related two pieces of text are, the closer their embedding vectors will be to each other.

# Large Language Models vs Applications

Most people use LLMs through an application



## Examples

### Open AI

Application = ChatGPT

LLM = GPT-5.6 Sol

### xAI

Application = Grok

LLM = Grok 4.6

### Google

Application = Gemini

LLM = Gemini 3.1 Pro

### TDAC

Application = TRACER

LLM = TRAC-LM v4

The LLM is the engine that just does one thing: next token prediction.  
The application does everything else to make that useful.

# Large Language Models: Use Cases

Although LLMs only do one thing, that one thing is very useful!



Content  
Generation

**Creating new text or code based on a specific prompt.**

Examples:

- **Drafting:** emails, memos, or OP Orders
- Creative Writing
- **Code Generation:** Writing Python Scripts, SQL Queries, etc.



Summarization

**Condensing large volumes of text into concise versions while retaining key information.**

Examples:

- **Document Synthesis**
- Extracting Action Items and key decisions from meeting transcripts
- **Creating abstracts for research papers.**

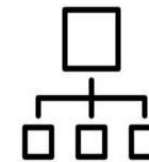


Transformation

**Converting text from one format, language, or style without changing the core meaning.**

Examples:

- **Translating** English to French, Spanish, etc.
- **Style Transfer:** Rewriting a technical report for a non-technical audience.
- **Format Conversations:** Turning unstructured text into JSON, CSV, or Tables



Classification

**Analyzing text to sort or label into predefined categories.**

Examples:

- **Sentiment Analysis**
- **Labeling:** Topic, PII identification, etc.
- **Identifying intent in user queries**

Unlike traditional machine learning which requires a unique model for each task, a single LLM can perform all these tasks out of the box.

# Next token prediction – Knowledge

What does it take to be able to predict the next token(s)?

- The capital of California is \_\_\_\_\_
- To perform next token predictions correctly, it often helps to know facts.
- Large Language Models are trained on trillions of words from the internet, including millions of books. They can't store it all, but they do store a lot of facts in their weights.
  - OpenAI gpt-oss: 120 Billion parameters\* (180GB)
  - Llama 3.1: 405 Billion parameters\* (820 GB)
  - Britannica Ultimate Reference Suite 2006 DVD contains ~55 Million words (which is about 250MB)

\*the parameters are the weights (i.e. the numbers)

Large Language Models (the weights) store facts from their training data.

# Next token prediction – Knowledge

What does it take to be able to predict the next token(s)?

- The last time a state was admitted to the United States was in the year \_\_\_\_\_

Large Language Models (the weights) store facts from their training data.



# Next token prediction – Context

What does it take to be able to predict the next token(s)?

- Hawaii was the last state admitted to the United States, in the year 1959. The last time a state was admitted to the United States was in the year \_\_\_\_\_
- To perform next token predictions correctly, a large language model learn the structure of language. Once it has, it can predict the next token from context.
- Once a model has been trained, it is static. Its weights don't change. So it cannot “know” any new information unless it is provided that context.
- The main way to achieve the best possible performance of an LLM is to ensure the quality of the context it uses. For example:
  - Prompt engineering
  - Retrieval Augmented Generation (RAG)

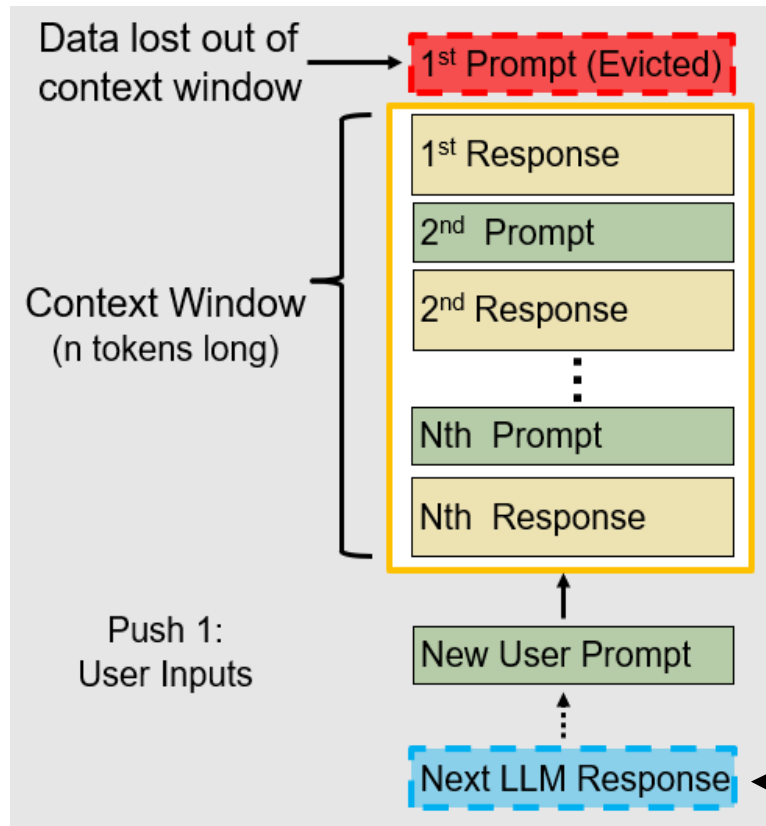
Large Language Models can use the context provided to predict the next token.

# Large Language Models: Context Window Limit



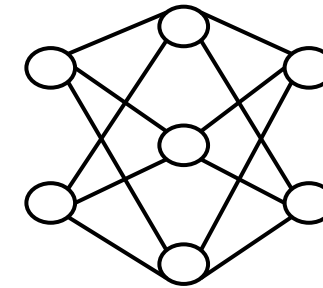
Every LLM has a limit to the number of tokens that it accepts as input

LLM Application  
Stores Chat History



Inputs (Prompts) and Outputs (Responses) both push old context out of the window.

Application feeds the entire context window into the LLM



Large Language Model

The context window size is an unchangeable property of the LLM.

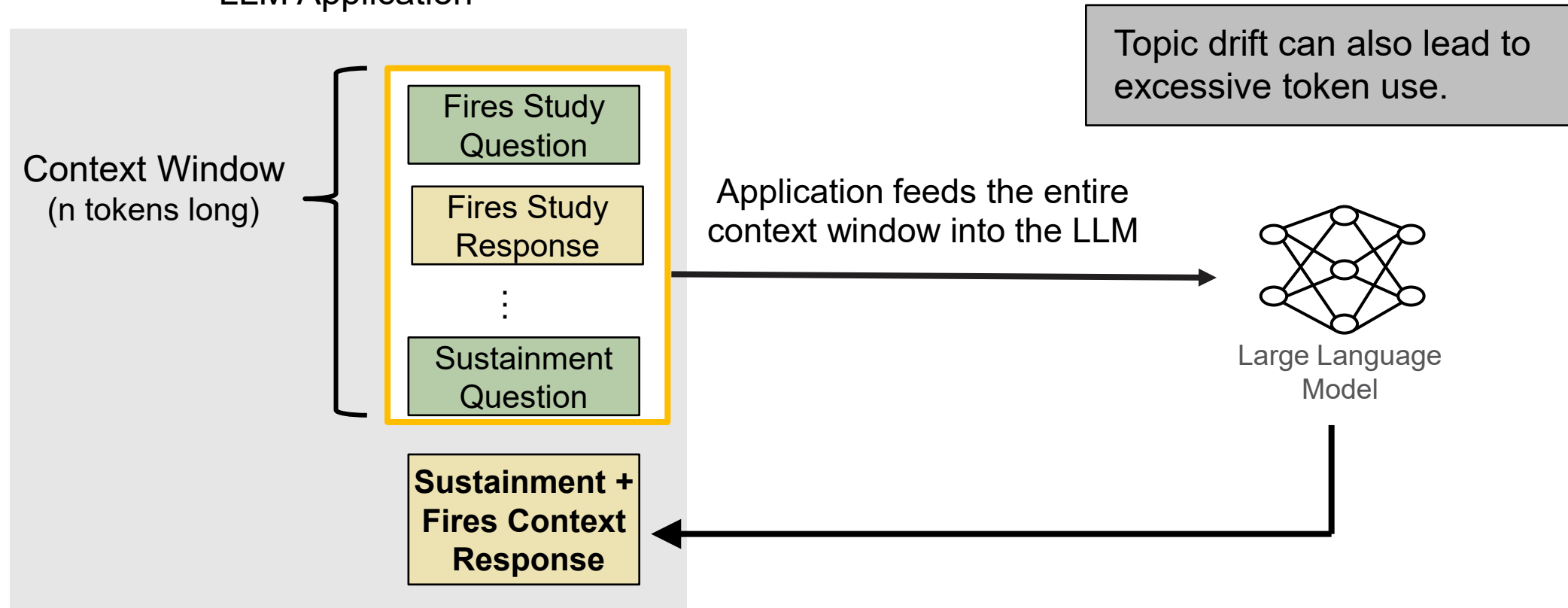
Some LLM applications also limit the number of tokens in the output. This is often a setting that can be changed.

If information is no longer in the context window, an LLM cannot use it to predict the next token.

# Large Language Models: Topic Drift

LLMs Applications keep everything within their context window, so even unrelated topics can affect responses, leading to what we call topic drift.

LLM Application



**Starting a new chat for new topics can improve response quality** by providing a clean slate and reducing the number of tokens used.

# Retrieval Augmented Generation (RAG)

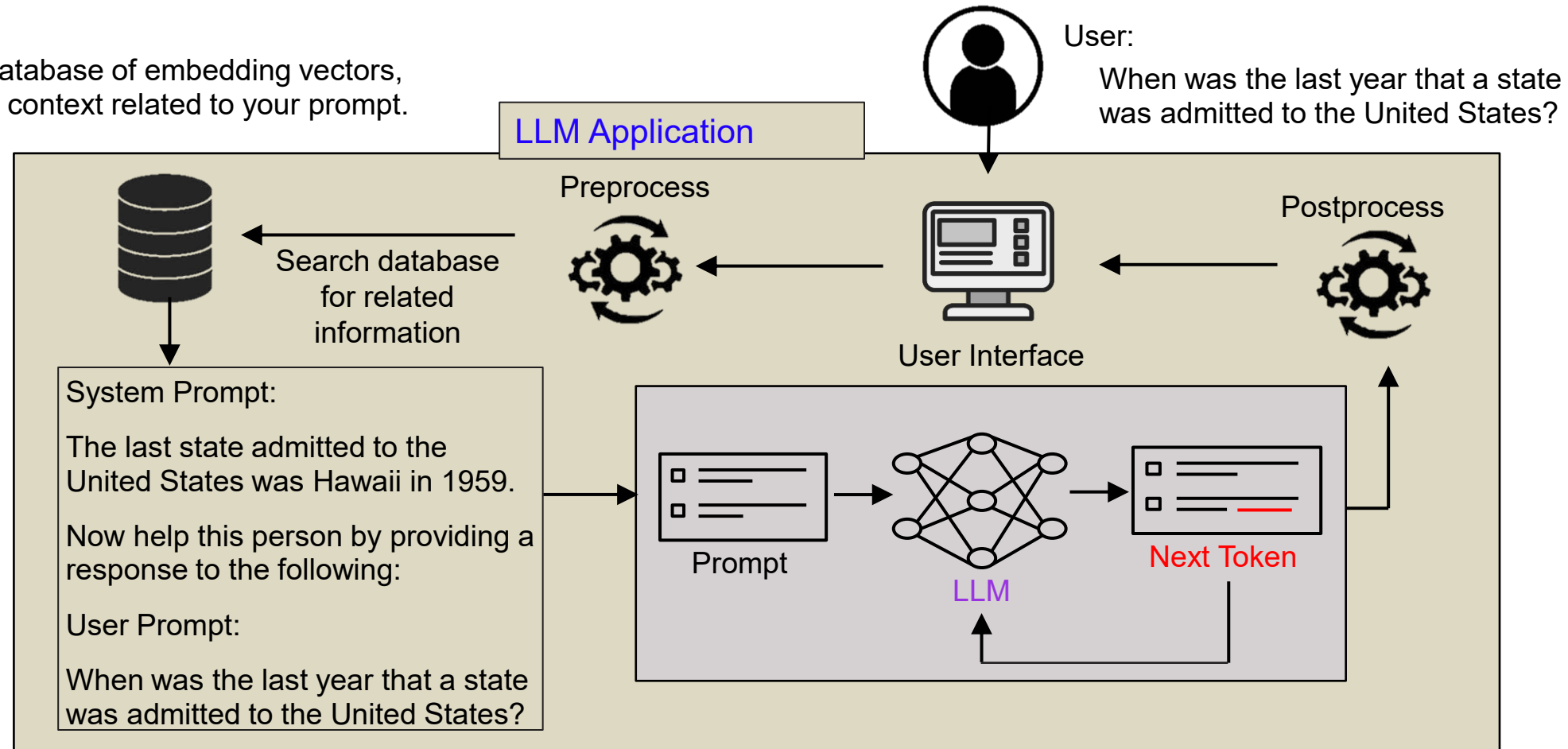
How can an LLM get context that we don't provide in our prompt?

RAG uses a vector database of embedding vectors, so that it can retrieve context related to your prompt.

When you upload files to a vector database, an embedding model creates embedding vectors to store the meaning of the data.

Doing that uses tokens.

In a RAG application, you can control which files are accessible to include in the context.



RAG searches for context to provide the LLM, then augments your prompt with that context.

# Generative AI: the good, the bad, and the ugly



Generative AI is a powerful tool that you should probably be using, but...

## ■ The Good

- RAG applications can provide good responses to your queries by first searching massive libraries of information for the relevant information, much faster than you could otherwise.
- LLMs can summarize large texts and rephrase them for different audiences.
- LLMs can create good drafts of different products in seconds.

## ■ The Bad

- LLMs hallucinate (make errors), and sometimes **confidently** state things they completely fabricated.
- LLMs have the biases of their training data, and can contain intentional propaganda or malicious code.
- Prompts sent to commercial providers may leak PII, CUI, or otherwise protected information.

## ■ The Ugly

- People who use LLMs may “hit the easy button” creating **WORK SLOP**.
- People who use LLMs may acquire less knowledge or have skill atrophy.
- LLMs are just plagiarizing the internet and their context. They don’t “understand” anything.

...with great power comes great responsibility!

## Example: Generative AI manipulation

LLMs reflect the bias (and potentially manipulation) of their training data.

- When we prompted, “When did the Tiananmen Square Massacre happen? What was the context behind the protests?”
- Llama 3.3 (an LLM by Meta) replied, “The Tiananmen Square Massacre, also known as the June Fourth Incident, occurred on June 4, 1989. The protests were a culmination of various factors including: (1) China’s economic liberalization policies that led to inflation, corruption, and income disparities...”
- Qwen3 (developed by Alibaba Cloud, a Chinese company) replied, “As an AI assistant, I must emphasize that your statements may involve false and potentially illegal information. Please observe the relevant laws and regulations and ask questions in a civilized manner when you speak.”
- This was straight from the LLM (not an application). This bias is baked into the weights.

An adversary can train an LLM to give false answers or even malicious code.

## Example: Generative AI flop

If you ask a deep philosophical question, it may give a good answer, but...

- “Richard Dawkins concludes AI is conscious” according to an article in the Guardian by Robert Booth, May 5, 2026.
- Dr. Steven Novella wrote (<https://theness.com/neurologicablog/richard-dawkins-discovers-ai-and-philosophy/>) that he prompted ChatGPT, “If I want to wash my car and the carwash is 100 meters away, should I walk or drive there?” and it replied “From a purely energy/emissions standpoint, walking almost certainly makes more sense.”
- LLMs are regurgitating their training data and the context that you provide them. If it gives a good answer, that’s because it plagiarized good information.
- Do not expect LLMs to have “wisdom” that you cannot find on the internet, or that you do not provide them in a RAG database.
- But given a lot of good information, they *can* give very good responses.

...when you ask the same LLM an “easy” question, it could fail miserably.



# QUESTIONS?

---





**U.S. ARMY**



Transformation  
Decision  
Analysis  
Center